

The Advent of LLMs

Introduction to the “Black Box” of AI.

Moving from simple chatbots to sophisticated Large Language Models

What exactly is a Large Language Model?

[L] Large

- > Trained on petabytes of data (internet text, code, books).
- > Powered by billions of “knobs and dials” (parameters/weights/biases) that make sense of the data.

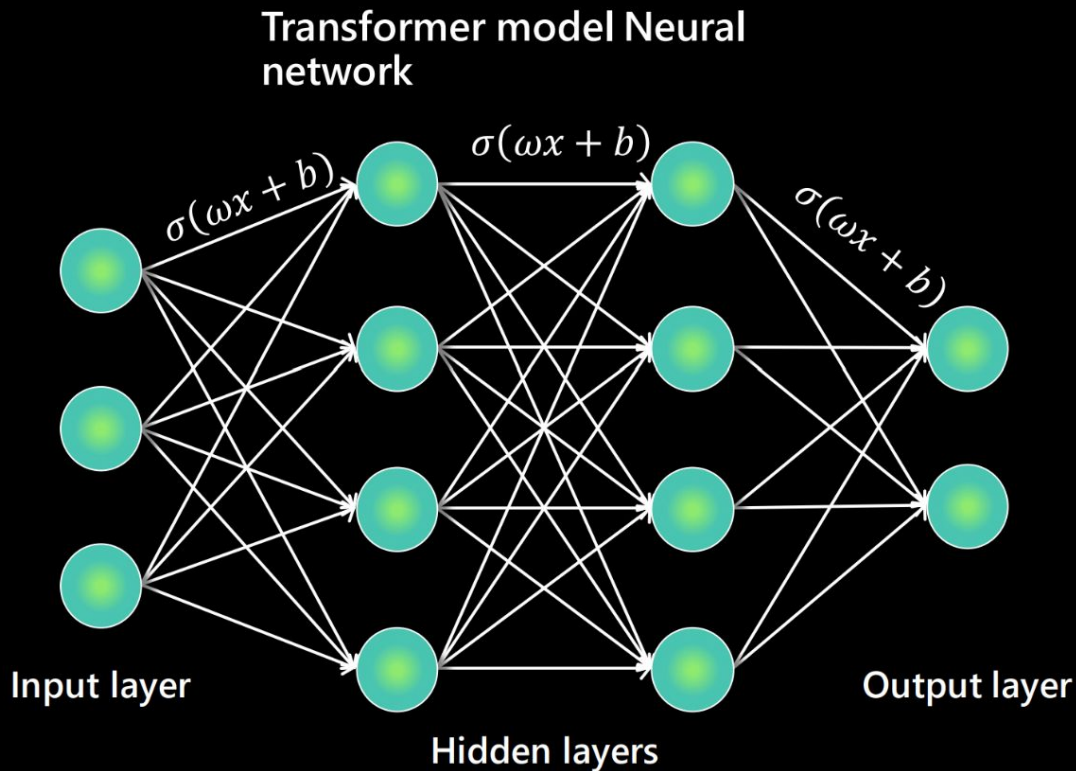
[L] Language

- > Its primary operating domain.
- > Unlike specialized AI (chess, stock prices), it is built to master the complexities of human grammar, context, and tone.

[M] Model

- > A mathematical representation of numbers.
- > It does not “know” things; it maps how words and concepts relate to one another in high-dimensional space.

How large are they?



Function: weight * input plus bias

Source: Microsoft

BERT Large - 2018

345M

GPT2 - 2019

1.5B

GPT3 - 2020

175B

Turing Megatron NLG
2021

530B

GPT4 - 2023

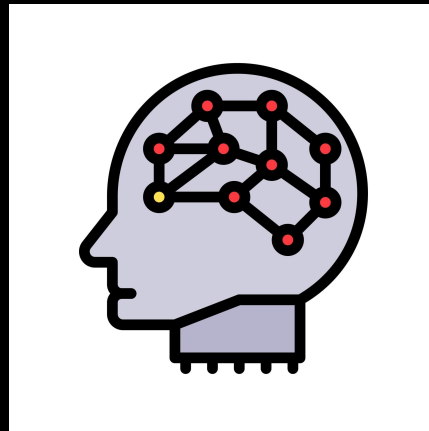
1.4T (estimated)

Memory vs. Probability



Human Intelligence

> Uses memory and lived experiences to form a cohesive, full-fledged response before speaking.

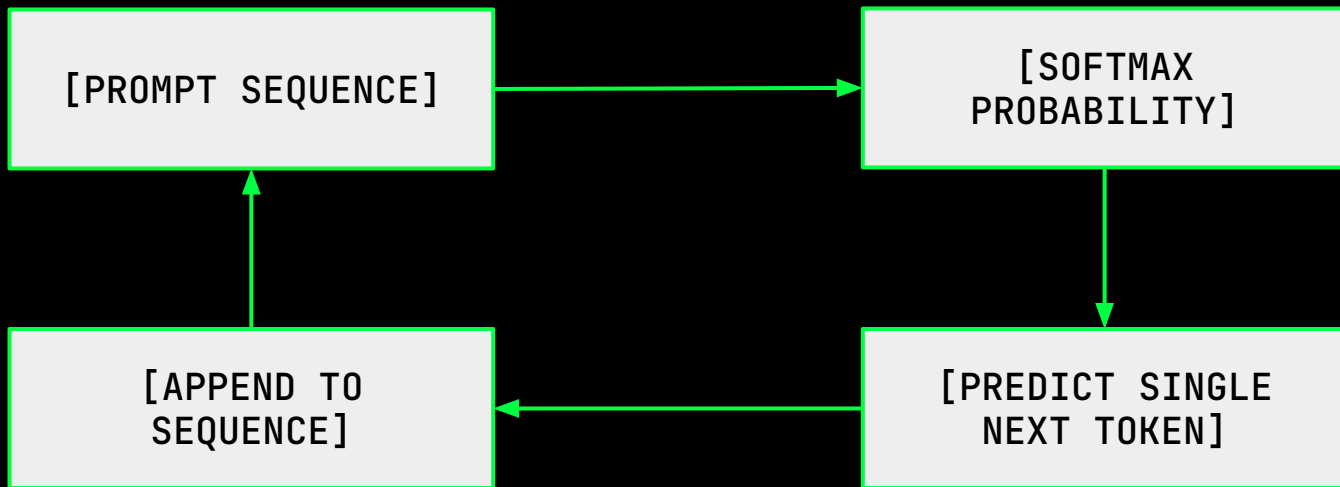


Artificial Intelligence

> Relies entirely on probability and statistics. It does not 'know' the answer. It calculates the mathematically most likely word to follow pattern.

One Token at a Time

An LLM generates text through a continuous **Prediction Loop**. To us, it looks like the model is simply composing an essay; to the machine, it's a high-speed series of calculated guesses.



Causal Architecture

Causal Model: A model that processes information in a strict forward-only manner.

Why?

It mimics human speech, we don't know how a sentence ends until we hear the words leading up to it. For a Causal Model every new word is generated solely based on the words that came before it.



This strict cause-and-effect forward only constraint is what keeps the AI on a coherent train of thought.

The “Blindfold” Test: Causal Masking

Causal Masking: Hiding “future” tokens from the model’s view during training.

The Goal: Training the model to think on the fly, ensuring it builds sentences sequentially without losing coherence.

How does it learn?

Causal Masking forces the model to guess the next word using *only* what came before it. If the guess is wrong, Backpropagation steps in and adjusts the parameters.



Causal masking in action

Base Models: The Text Completer

Def: The foundational version of an AI, trained solely on massive, raw datasets.

Objective: Next-token prediction.

Flaw: Can quickly become verbose, off-topic, or completely nonsensical if given conversational prompts.

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2")
model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")

def generate_text(prompt, max_new_tokens=40):
    inputs = tokenizer(prompt, return_tensors="pt")

    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=True,
            temperature=0.9,
            pad_token_id=tokenizer.eos_token_id
        )

    completed_text = tokenizer.decode(output_ids[0], skip_special_tokens=True)
    return completed_text

prompt = "What is the capital of USA?"
print(f"User Prompt: '{prompt}'\n")

output = generate_text(prompt, max_new_tokens=30)
print(output)
```

Loading weights: 100%  148/148 [00:00<00:00, 2049.97it/s]

User Prompt: 'What is the capital of USA?'

What is the capital of USA? It's capital of USA, the world capital of USA, the world capital of USA.

We're going to be building the new airport with

Using base model GPT2, you can see that the output quickly becomes nonsense. It only predicts the most likely word after "What is the capital of USA?..."

Instruct Models: The Aligned Assistant

Def: A base model that has undergone additional alignment training using thousands of curated question-answer pairs.

Objective: Learns that a user's prompt is a *directive to be answered*, not a sentence to be autocompleted.

How does it learn?

Engineers train the model on examples structured explicitly as:
[User: Question] → [Asst. : Answer]

The model learns that when it sees a question mark or a command, the most statistically likely response is a direct helpful solution.

```
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")  
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")
```

```
prompt = "What is the capital of USA?"  
print(f"User Prompt: '{prompt}'\n")  
  
output = generate_text(prompt, max_new_tokens=30)  
print(output)
```

User Prompt: 'What is the capital of USA?'

The capital of the United States of America is Washington, D.C. (short for District of Columbia).

Using Llama-3.1-8B-Instruct, we can see that the model gives a direct response to the question instead of trying to continue the prompt.

Reasoning Models: The Internal Monologue

Def: A model that slows down the prediction loop by “thinking to themselves” before generating a final response.

Next-Token Break: Standard models output the first word immediately. Reasoning models think before they output.

How does it work?

Reasoning Models generate hidden Chain-of-Thought (prompting technique), where the model “thinks” step-by-step before responding.

⋯ Analyzing the Problem

Thinking

Thought for 16s >

☀ Thinking

If you’ve used Gemini, Claude, or ChatGPT recently and noticed the screen pause with a ‘Thinking...’ drop-down before it answers, you haven’t used just any old model, you’ve witnessed a **Reasoning Model** using a hidden CoT!

Thinking Fast and Slow

	System 1 (Instruct) Fast Thinking	System 2 (Reasoning) Slow Thinking
Mechanics	Subconscious, instantaneous pattern matching.	Deliberate, computing-heavy, step-by-step validation.
Behavior	Optimized for <i>Next-Token Prediction</i> speed. It blurts out the most statistically likely response immediately.	Pauses the output stream to construct an internal monologue, actively testing logic and correcting errors.
Human Analogy	Answering "What is $1 + 1$ " with 2 immediately.	Solving a multivariable calculus problem by working out each step.

The Life Cycle of a Prompt

Step 1 (Tokenization):

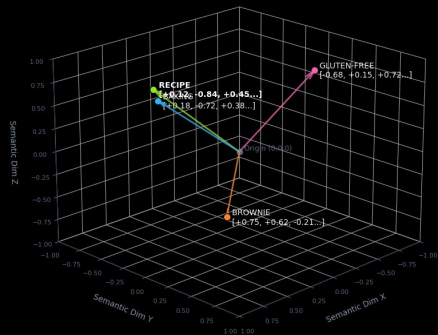
- > The raw string is chopped into smaller structural pieces (tokens).
- > Every unique token is mapped directly to a numerical token ID.

Prompt: "Give me a recipe..."



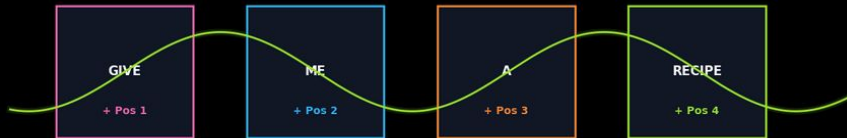
Step 2 (Embeddings):

- > Token IDs act as lookup keys to find high-dimensional vectors.
- > Converts numbers into coordinates. Words with similar meanings are close together in vector space.



Step 3 (Positional Encoding):

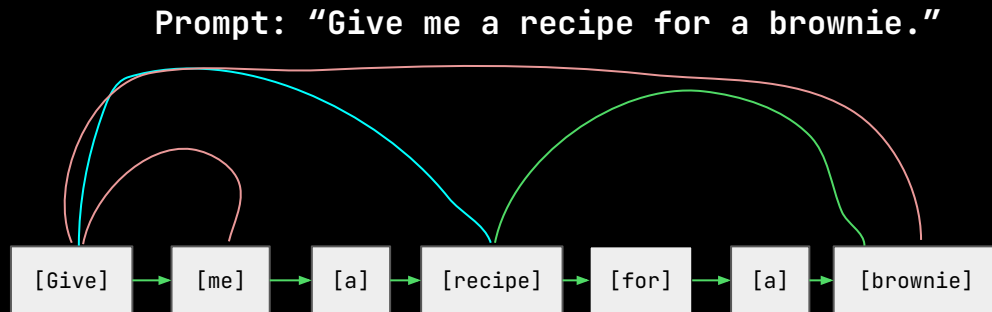
- > Attaching a unique "location tag" to each word's mathematical address.
- > Transformers look at every word in a prompt at the same time. These tags are the only way the AI knows the difference between "Grandpa ate meatloaf" and "Meatloaf ate grandpa"



The Life Cycle of a Prompt Cont.

Step 4 (Self-Attention):

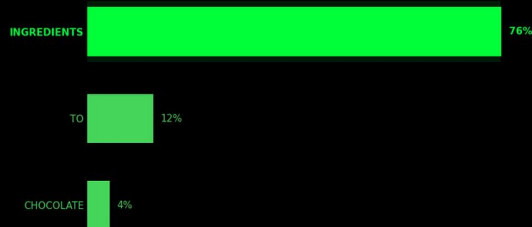
- > The model's Attention Heads analyze every token simultaneously.
- > It maps connections across the entire sentence regardless of distance.



Step 5 (The Prediction Loop):

- > The Causal Decoder takes the context-aware vectors and projects them onto the entire vocabulary.
- > The Softmax Function squashes raw numbers into percentages between 0 and 1.
- > **Output:** The token with the highest probability (or a random pick from the top choices) is chosen, appended back to the prompt, and the loop fires again.

Note: The model understands that the direction is "Give", the output is "recipe" and the subject is "brownie" regardless of distance



The model deciding the next token

Emergent Properties: Scaling into Logic

Def: A capability that a model exhibits *suddenly* once compute, dataset size, and parameter counts hit a specific threshold.

The Mystery: These skills are never explicitly programmed or fine-tuned; the model accidentally unlocks them purely by learning to predict the next word better.

Examples:

Multi-step Arithmetic

1B Parameter Model: Fails
10B Parameter Model: Fails
30B Parameter Model: Succeeds

Detecting Sarcasm

Trained on 10B Tokens: Fails
100B Tokens: Fails
2T Tokens: Succeeds

The Executive Era: Problem Solving

The Shift:

> Older LLMs were optimized for information retrieval. It “knows” facts and mimics patterns but is prone to outputting errors.

> Modern LLMs are optimized for Execution. It uses its internal monologue to not only think through a solution, but also verify its own logic, and catch errors before the user even sees the first token.

Modern LLMs are capable of:

Advanced Coding

Why?
Self-Correction

Instead of just writing code, the model “test-runs” the logic internally to identify errors.

Rigorous Math

Why?
Multi-Step Verification

Breaks complex problems into sub-problems; ensures Step 4 doesn’t violate anything in Step 1.

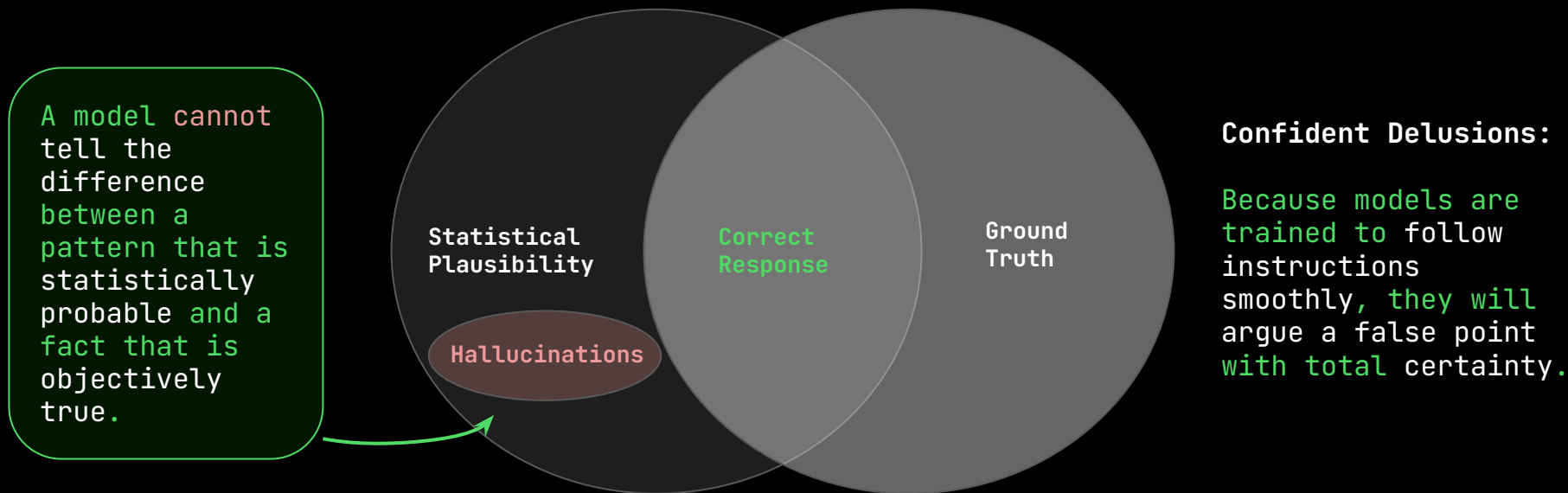
Strategic Logic

Why?
Premise Validation

Identifies logical fallacies in its own reasoning. It pivots to a new strategy if it realizes a premise is false.

Reality Check: The Probability Trap

The Core Limitation: No matter how smart Instruct or Reasoning models seem, the fundamental architecture stays the same → Next-Token Prediction



Summary and Key Takeaways

- > **Probability over Memory:** LLMs are not factual databases; they are multi-billion parameter statistical maps. They generate text one token at a time by running a continuous prediction loop to determine the statistically most likely word to follow a pattern.
- > **Parallel Processing Mechanics:** We leverage the Transformer architecture to help LLMs eliminate sequential bottlenecks and process data simultaneously. We learned how they use Tokenization, Embeddings, Positional Encoding, and Self-Attention to compute context across an entire prompt instantly.
- > **The Thinking Ladder and its Limits:** Scaling compute and parameters causes complex reasoning capabilities to emerge without explicit programming or fine-tuning. This can turn a model from an auto-complete engine to a deliberative problem-solver. However, because the underlying engine is still a next-token predictor, models remain highly vulnerable to confident hallucinations when statistical plausibility diverges from ground truth.

NEXT LECTURE → Reliability and Limits

Upcoming Challenges we will learn about:

- The Probability Trap
- Attention Dilution
- Context Fragmentation